JWT Security Best Practices

For JWT

- Verify Signature
- Validate Expiry
- Use RS256 (Public/Private Keys)

For Refresh Tokens

- Hash before storing
- Rotate every use
- Detect reuse
- Set expiry
- Store in HTTP-only cookies

---

1. Verify Signature

Suppose someone changes the JWT payload.

Original payload:

```
{
  "sub": "42",
  "role": "USER"
}
```

Attacker modifies it:

```
{
  "sub": "42",
  "role": "ADMIN"
}
```

If the server simply decodes the JWT and trusts it:

❌ The attacker becomes an admin.

That's why the server must verify the signature.

Verification means:

```
Expected Signature == Signature inside JWT
```

If they don’t match:

```
401 Unauthorized
```

---

2. Validate Expiry

JWT payload contains:

```
{
  "exp": 1710000900
}
```

`exp` = expiration timestamp.

Example:

- Login: **9:00 AM**
- Token expires: **9:15 AM**

At **9:20 AM**:

```
Current Time > exp
```

Reject it.

Why?

If an attacker steals the token:

Damage is limited.

Instead of:

```
Token valid forever ❌
```

---

3. Use RS256 (Public/Private Keys)

This is one of the most important production practices.

What is RS256?

RS256 means:

```
RSA Signature with SHA-256
```

It is an **asymmetric signing algorithm**.

JWT Header:

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

Symmetric vs Asymmetric

HS256

Uses **ONE secret**.

```
SECRET
   │
   ├── Sign JWT
   └── Verify JWT
```

Example:

```
Secret = "mysecret"
```

Everyone verifying tokens knows the same secret.

RS256

Uses **TWO keys**:

- Private Key
- Public Key

Flow:

```
Private Key → Sign JWT
Public Key  → Verify JWT
```

Real Example

Suppose your company has:

- Auth Server
- Order Service
- Payment Service
- Profile Service

Using HS256

All services have:

```
SECRET_KEY
```

```
Auth Service      SECRET
Order Service     SECRET
Payment Service   SECRET
Profile Service   SECRET
```

Problem:

If Payment Service is hacked:

Attacker steals `SECRET_KEY`

Now attacker can generate:

```
{
  "sub": "1",
  "role": "ADMIN"
}
```

Sign it.

Every service trusts it.

Entire system compromised.

Using RS256

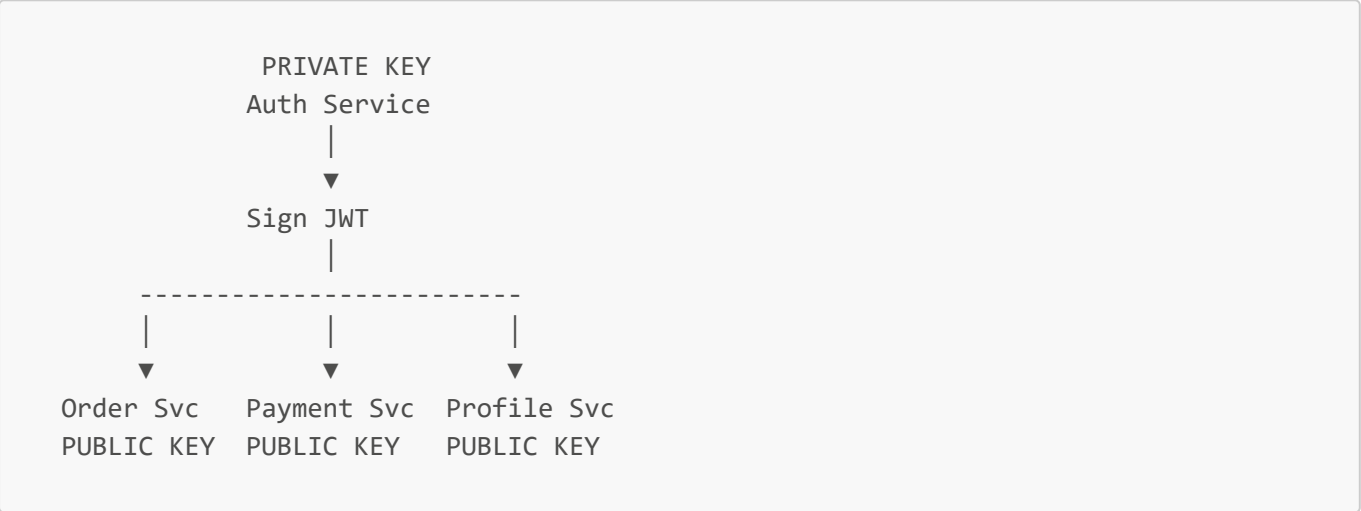
Only Auth Service has:

```
PRIVATE KEY
```

All other services have:

```
PUBLIC KEY
```

Architecture:



If Payment Service is hacked:

Attacker gets:

```
PUBLIC KEY
```

Can they verify JWTs?

✅ Yes

Can they create JWTs?

❌ No

Because only the private key can sign.

Why SHA-256?

RSA signs the data.

SHA-256 creates a digest (fingerprint) of:

```
header.payload
```

Then RSA signs that digest.

So:

```
RS256 = RSA + SHA256
```

---

4. Hash Refresh Tokens Before Storing

Treat refresh tokens like passwords.

Suppose refresh token is:

```
RT123ABC
```

Bad approach:

Database stores:

```
RT123ABC
```

If DB leaks:

Attacker uses it directly.

Better:

Store:

```
SHA256(RT123ABC)
```

Database:

```
A7C9F...
```

When client sends refresh token:

```
Hash incoming token
↓
Compare hashes
```

If DB leaks:

Attacker cannot directly use tokens.

---

5. Rotate Every Use

Suppose refresh token:

```
RT1
```

Client uses RT1.

Server returns:

- New Access Token
- New Refresh Token (RT2)

And invalidates RT1.

Flow:

```
RT1 → RT2 → RT3 → RT4
```

This is called:

Refresh Token Rotation

Why Rotate?

Suppose attacker steals RT1.

Legitimate User:

Uses RT1.

Gets RT2.

Attacker:

Later uses RT1.

Server says:

```
RT1 was already used.
```

Suspicious.

Attack detected.

---

6. Detect Reuse

Database:

```
RT1 → Used
RT2 → Active
```

Attacker sends:

```
RT1
```

Server detects:

```
Refresh Token Reuse
```

Action:

```
Invalidate entire session
Force login again
```

This protects against stolen refresh tokens.

---

7. Set Expiry

Refresh tokens should expire too.

Example:

```
Access Token  → 15 minutes
Refresh Token → 7 days
```

Without expiry:

```
Refresh Token valid forever ❌
```

If stolen:

Attacker gets permanent access.

---

8. Store in HTTP-Only Cookies

Example:

```
Set-Cookie:
refreshToken=xyz;
HttpOnly;
Secure;
SameSite=Strict
```

HttpOnly

JavaScript cannot read it.

Bad:

```
document.cookie
```

Cannot access:

```
refreshToken
```

Why?

Suppose website has XSS:

```
<script>
send(document.cookie)
</script>
```

Without HttpOnly:

Attacker steals refresh token.

With HttpOnly:

Refresh token remains hidden.

Much safer.

Putting It All Together

Production-grade JWT flow:

```
User Login
    │
    ▼
Auth Server
    │
    ├─ Verify credentials
    ├─ Sign Access Token using RS256
    ├─ Generate Refresh Token
    ├─ Hash Refresh Token before DB storage
    └─ Send:
         Access Token (15 min)
         Refresh Token (HttpOnly cookie)
              │
              ▼
Client uses APIs
              │
              ▼
Access Token expires
              │
              ▼
Use Refresh Token
              │
              ▼
Rotate Refresh Token
              │
              ▼
Detect reuse if old token appears
              │
              ▼
Expire refresh token after 7 days
```

Summary

Practice	Why
Verify Signature	Prevent forged JWTs
Validate Expiry	Limit damage from stolen tokens
RS256	Only Auth Service can issue tokens
Hash Refresh Tokens	Protect against DB leaks
Rotate Refresh Tokens	Reduce replay attacks
Detect Reuse	Catch stolen refresh tokens
Set Expiry	Avoid permanent access
HTTP-only Cookies	Protect tokens from XSS attacks

The Big Picture

```
Verify Signature → Prevent forged JWTs
Validate Expiry → Limit damage from stolen tokens
RS256 → Only Auth Service can issue tokens
Hash Refresh Tokens → Protect against DB leaks
Rotate Refresh Tokens → Reduce replay attacks
Detect Reuse → Catch stolen refresh tokens
Set Expiry → Avoid permanent access
HTTP-only Cookies → Protect tokens from XSS attacks
```